

MASTER'S THESIS

Personalized Product Ranking for B2B Marketplaces

by

Aditya Marada

(STUDENT NUMBER: 2844807)

*Submitted in partial fulfillment of the requirements
for the joint UvA-VU degree of
Master of Science
in
Computer Science
at
Vrije Universiteit Amsterdam
and
University of Amsterdam*

July 14, 2026

Certified by

Shahab Ud Din
First Supervisor

Certified by

Michael Holmes & Michael Garstka
Daily Supervisors

Certified by

Jeewanie Jayasinghe Arachchige
Second Reader

Acknowledgements

This thesis would not have been possible without the guidance of Professor Shahab Ud Din whose thoughtful and constructive feedback helped me refine and polish my project. I am also deeply indebted to my supervisors Michael Holmes and Michael Garstka for the time, patience and support they devoted to this project. I will always cherish the insightful conversations we shared. I have learned so much from you all, and as I conclude this thesis I have a much greater appreciation for all you have taught me.

I would also like to acknowledge Qogita and the entire team, who generously offered me this opportunity and consistently encouraged me throughout this work.

I am deeply grateful to my mom Usha, my dad Venkateshwar and my sister Shruti for their unconditional love and support. I am also thankful for my friends, the old and new ones I found along the way for always listening to me and providing the moral support that helped this thesis materialize.

Finally, I would like to thank Dipti for her love, care and encouragement that made this thesis a reality.

*Aditya Marada
Amsterdam
July 2026*

Personalized Product Ranking for B2B Marketplaces

Aditya Marada
Vrije Universiteit Amsterdam
Amsterdam, NL
a.v.marada@student.vu.nl

ABSTRACT

Personalized ranking systems have advanced rapidly, yet the benchmarks and methods that drive this progress assume a consumer setting: the user is an individual, the conversion signal is a small-volume purchase, and the catalog is not offer-competitive. Business-to-business wholesale marketplaces challenge each of these assumptions. Buyers are organizations placing bulk orders after multi-session deliberation, purchase events are rare and delayed, and the same product may be listed by competing suppliers at varying prices and stock levels. No impression-level dataset or ranking system designed for this setting has been published.

In this work we construct such a dataset from scratch. We build a seven-month impression-level dataset from production analytics events at Qogita, a B2B wholesale marketplace, covering 163,000 logged-in buyers and 450,000 products with every feature constructed point-in-time through a reproducible pipeline and purchase outcomes linked via a deterministic cart-trace chain. We train a LightGBM LambdaMART ranker over 78 features spanning product popularity, buyer-product affinity, session context, and behavioral and semantic embedding similarities, selecting the label-gain map and hyperparameters empirically on a development subset.

The ranker improves NDCG@10 by 9.55 percentage points over the logged production order on the full 52-million-row test window. The lift holds across all four catalog surfaces, degrades gracefully for cold-start buyers, and the tuning configuration transfers without modification to the model trained on up to six times as much data. These results show that a personalized listwise ranker can be built for a B2B setting and deliver substantial, robust improvements over the logged production order.

1 INTRODUCTION

In every machine learning-based, user-facing recommendation system, there is an underlying objective, and this objective is fundamentally linked to the notion of the user. In most cases, the output of a system is to satisfy an objective that is a function of the *user*. The user is in turn inferred from the interaction logs before the recommendation event and evaluated by the logs that follow (e.g., clicks, purchases, likes) [26]. In this work we investigate a ranking system for a B2B marketplace, where the user represents an organization rather than an individual consumer.

B2B marketplace users are primarily motivated by commercial constraints and organizational procedures. This is in contrast to users in a B2C context where personal tastes play a greater role in product discovery. Importantly, the individual placing the order is not the same as the entity whose preferences we wish to model where the organization’s purchasing patterns are a collective signal, accumulated across users, categories, and time. This collective, commercially-grounded nature of B2B demand means that the standard framing of ranking items by their relevance to a

user’s latent preference might be insufficient. There remains a gap in the literature for evaluation of a system that ranks by fit to an organization’s commercial profile: its category affinities, its typical basket composition, and the trajectory of its ordering behavior over time.

The problem of ranking has been studied deeply under the umbrella *learning-to-rank* (LTR), and the literature provides us with a rich toolkit [18]. LambdaMART [4] has become the most widely used generic listwise approach. The algorithm computes pairwise lambda weights scaled by the corresponding change in NDCG that a document swap would induce, and uses these to train a gradient-boosted ensemble of trees. This method performs strongly on standard LTR benchmarks [22] and has been deployed at large scale for web search [18] and B2C marketplace search [9]. This formulation in the form of gradient-boosting (1) makes it robust to noisy and sparse labels, (2) handles mixed feature types natively and (3) is interpretable enough to audit in a large-scale production setting.

The limitation of the existing evaluations of this algorithm is that all of this validation has been done mostly on consumer data. The benchmarks are datasets derived based on user interactions in a B2C setting. For example, LETOR [22] and the Yahoo! Learning to Rank Challenge [5] derive relevance from web-search clicks issued by individual users, and the Expedia Personalized Sort dataset [7] treats hotel booking as a single-transaction consumer purchase event. The feature designs assume a consumer user and the conversion signal is single-individual consumer in nature.

1.1 Differences between B2C and B2B ranking

Table 1: B2C ranking assumptions vs. B2B ranking reality.

Dimension	B2C	B2B / Qogita
User	Individual	Organization
Conversion	Click / single purchase	Bulk order
Decision cycle	Short	Multi-session
Catalog	Product-level	Offer-competitive

Table 1 summarizes the main dimensions on which B2B marketplace ranking departs from standard B2C assumptions. Although some learnings and algorithms transfer from B2C to a B2B domain, a few assumptions about consumer behavior, motivations and requirements do not hold true in a B2B context.

Difference in user persona Any activity logging for a buyer account in a B2B setting is a combination of multiple individuals. This is true in two different ways: (1) a buyer account could be managed and operated by multiple individuals who have different browsing behavior, (2) the intent of the organization is different from the intent of a simple consumer browsing in e-commerce.

Conversion in a B2B context. A purchase is of large commercial value, follows multi-session deliberation and is rare relative to the number of impressions. A buyer might browse many products over several days, focusing on potential profit margins, but ultimately purchase only one or two.

Offer competitiveness. The same product may be listed by multiple competing suppliers at different prices and stock levels. However, the ranked unit studied in this work is the product card, identified by product/GTIN, not an individual supplier offer. Offer competitiveness enters the model through product-level aggregates such as minimum offer price, offer count, price spread, and total inventory.

We construct a dataset with one row per impression, linking the product shown to the buyer with the listing event, the buyer’s profile and procurement history, and downstream purchases. All of this data is captured point-in-time. To our knowledge, no impression-level dataset or ranking evaluation for a B2B wholesale marketplace has been published.

1.2 Research questions

In this work we investigate three questions, each tied to a specific comparison reported in Section 5.

- RQ1.** Does a gradient-boosted LambdaMART ranker trained on B2B impression-level data with graded relevance labels (click, add-to-cart, purchase) improve NDCG@10 over the logged production order on held-out listing pages?
- RQ2.** If we train the ranker to value clicks, add-to-carts, and purchases differently, which training objective produces the best ranking under the same canonical evaluation metric, and does the gain map selected at development scale transfer to the full test window?
- RQ3.** Do hyperparameters tuned on a 10% sample of the training data transfer to models trained at larger data scales? Specifically, does the Gain + HPO configuration selected at a sample of the dataset consistently improve full-test NDCG@10 over the default configuration across all training-data regimes (10%–60%), without requiring re-tuning at each scale?

1.3 Contributions

This thesis makes four contributions.

- (1) **An impression-level dataset for B2B marketplace ranking.** We construct a seven-month impression-level dataset from production event streams, with graded relevance labels attributed by a cart-traced chain that links listing impressions to purchase outcomes. The attribution and feature logic is implemented in a reproducible data pipeline that materializes the training dataset and enforces strict point-in-time correctness, ensuring that no feature contains information that became available after the corresponding impression.
- (2) **A LambdaMART ranker that improves over the logged production order.** We train a LightGBM LambdaMART ranker over features spanning product popularity, buyer–product affinity, session state, and target-encoded taxonomy. On the fixed held-out test set, the champion reaches NDCG@10 of 0.543 against 0.448 for the logged production order, a 9.55 pp lift. A training-fraction study over progressively larger subsets of the

same training window shows that test performance plateaus at the 30% training fraction; as shown in Figure 5, this identifies the operating point for deployment.

- (3) **An empirical label-gain selection for sparse B2B outcomes.** Table 5 lists five label-gain mappings we compare on a 10% subset of the data and select the linear mapping [0, 1, 2, 3] over the default purchase-weighted mapping [0, 1, 3, 7] by validation NDCG@10. This shows that the best training objective in this setting is not the steep purchase-weighted default, but a balanced gain map that extracts signal from clicks, add-to-carts, and purchases.
- (4) **A development-scale tuning protocol that transfers to larger training datasets.** After selecting the gain map, we run a hyperparameter sweep on the same development subset and carry the resulting configuration into larger training fractions without re-tuning. The gain + HPO configuration consistently outperforms the default across training fractions from 10% to 60% on the same held-out test set, improving NDCG@10 by 0.14–0.20 pp and demonstrating that small-sample tuning transfers to the larger training dataset.

1.4 Thesis structure

Section 2 reviews the domain terminology, learning-to-rank, ranking metrics, position bias, TOPSIS scoring, and the logged production order used as the evaluation baseline. Section 3 positions the work within the Learning-to-Rank (LTR) literature. Section 4 details the construction of the impression-level dataset, the point-in-time feature pipeline, the embedding design, the LambdaMART ranker, and the supporting infrastructure. Section 5 defines the experimental protocol and reports the baseline results followed by the training-fraction study that evaluates performance as progressively larger subsets of the training data are used. Section 6 interprets the results against the three research questions. Section 7 concludes the thesis.

Appendix A discusses ethical considerations, Appendix B reports AI usage during writing, Appendix C lists the champion model features, and Appendix D describes the submitted research artifact and reproducibility workflow.

2 BACKGROUND

2.1 Domain and Problem Setting

Qogita is a B2B wholesale marketplace where retailers and resellers purchase consumer goods from various suppliers. Buyers are typically SMBs to mid-market retailers who source inventory to resell to their own customers. A single purchase event can carry substantial commercial value because orders often span multiple product lines and follow longer deliberation than an impulse purchase.

Buyers discover products through four distinct listing surfaces, each with different intent characteristics.

- **Search.** A buyer enters an explicit query and results are retrieved by keyword and ranked by relevance. This surface carries the strongest purchase intent signal because the query encodes an explicit product need.

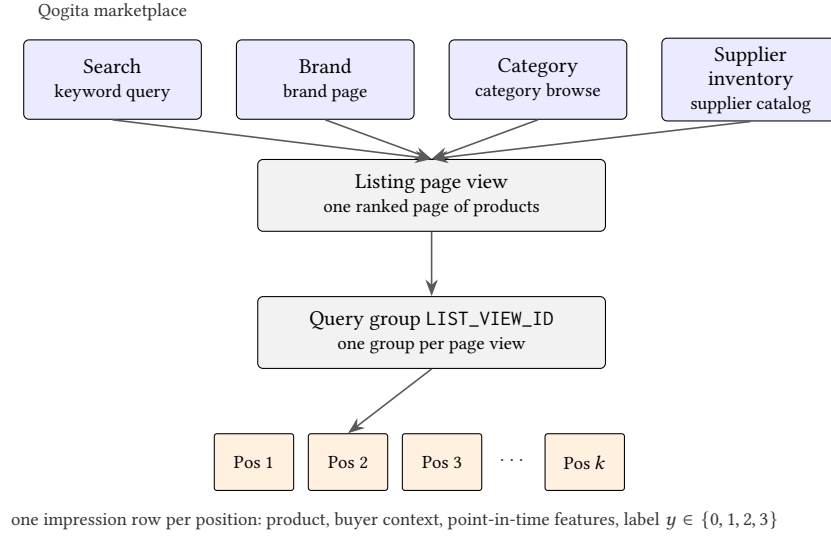


Figure 1: Four listing surfaces converge on the same ranking unit. Each page view is one query group (LIST_VIEW_ID); each product shown at a position within that view is one impression row used to train and evaluate the ranker.

- **Brand.** A buyer browses all products from a specific brand. Intent is narrower than search because the buyer has already committed to a supplier, making offer-level differentiation such as price and stock more decisive.
- **Category.** A buyer browses a product category (e.g. skin-care, confectionery). Intent is broader and exploratory as the buyer is assessing range rather than targeting a specific item.
- **Supplier inventory.** A buyer views the relevant subset of the catalog of a specific supplier.

As shown in Figure 1 each surface generates a *listing page*, which is an ordered list of products returned to the buyer. In this work, we aim to optimize engagement by showing relevant items higher up in the list where most user attention is concentrated.

2.2 Terminology

The following terms are used throughout this work.

Product and offer. A product is the catalog item being ranked, identified by a GTIN when available. An offer is a supplier-specific instance of that product with its own price, stock, and availability.

Surface. The entry point that produced a listing page: search, brand, category, or supplier inventory.

List view and query group. A list view is one rendered listing page. In the ranking model it is also the query group: the set of candidate products that must be ordered relative to one another.

Impression. One product shown at one position within a list view. The modeling dataset has one row per impression.

Candidate. A product available to be ranked within a list view.

Session. A sequence of marketplace interactions by the same buyer during a contiguous browsing episode.

Point-in-time feature. A feature computed only from information available before the impression timestamp, preventing future outcomes from leaking into training.

Cold-start. A segment where the model has little or no prior behavioral history for the buyer or product.

2.3 Learning-to-rank

Learning-to-rank (LTR) is the class of supervised machine learning techniques that train a scoring function to order items within a *query group* rather than predicting an absolute relevance score for each item independently [18]. Three families exist, distinguished by the loss function.

Pointwise. Each item in the group is treated independently and the model minimizes a regression or classification loss per item [18]. Pointwise models ignore the relative ordering of items within a group and are therefore suboptimal for ranking metrics such as NDCG.

Pairwise. The model minimizes a pairwise preference loss over pairs of items in the same group, as in RankNet [3]. This captures the ordering relation but weights all pair improvements equally, regardless of where in the list they occur.

Listwise. The model directly optimizes a list-level metric such as NDCG. LambdaMART [4] is one of the leading listwise methods. It approximates NDCG gradients by computing pairwise lambda weights that scale each pair’s gradient contribution by the NDCG change that swapping the two items would produce, then uses those weights to train a gradient-boosted tree ensemble. LambdaMART is consistently competitive on standard LTR benchmarks such as LETOR [22] and is the algorithm adopted in this work.

2.4 Evaluation metrics for ranked lists

NDCG@k. The normalized discounted cumulative gain at rank k is the ratio of the actual cumulative gain of the ranked list to the

ideal cumulative gain (the DCG of the perfect ordering):

$$\text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}, \quad \text{DCG}@k = \sum_{i=1}^k \frac{2^{y_i} - 1}{\log_2(i + 1)}, \quad (1)$$

where y_i is the relevance label of the item at rank i [12]. NDCG is bounded in $[0, 1]$, is independent of list length, and supports graded relevance, making it appropriate for the four-level label scheme used in this work.

MRR@k. The mean reciprocal rank of the first relevant item (label $y > 0$) in the top k . MRR@10 is more sensitive to whether a relevant item reaches the very top of the list than NDCG@10.

Hit@k. The fraction of query groups that contain at least one item with a relevant engagement in the top k . Hit@5, Hit@5_ATC, and Hit@5_BUY report coverage at the click, add-to-cart, and purchase level respectively.

2.5 Position bias in implicit feedback

Implicit feedback signals (clicks, add-to-carts) are biased by the position at which an item is displayed: an item shown at position 1 receives more exposure than one at position 10, even if the latter is equally relevant. Clicks are therefore affected by exposure bias, while add-to-cart events are additionally conditional on the user clicking through to the product detail page (PDP) which introduces a source of selection bias. If a model is trained directly on binary click labels without accounting for these biases, it learns a confound between the position and relevance [13].

In this work we use two lighter-weight mechanisms rather than full counterfactual training. First, graded relevance labels that reward add-to-cart and purchase more than clicks that reduce the dominance of the position-biased click signal. Second, position-weighted NDCG (W-NDCG@10) applies inverse-propensity weights estimated from the training split to up-weight impressions at lower positions which is used for evaluation.

2.6 TOPSIS

Technique for order of preference by similarity to ideal solution (TOPSIS) is a multi-criteria decision analysis method which scores each alternative by its normalized distance from a positive ideal solution and a negative ideal solution [11]. In the Qogita production system, the TOPSIS algorithm is used to compute a per-product score from available static product attributes, namely recent sales, availability, and price competitiveness. Since the score is computed at the product level, it is identical for every buyer meaning it does not incorporate buyer identity, purchase history, or session state. Products with indistinguishable attribute vectors receive the same score, creating large ties that are broken arbitrarily.

2.7 LightGBM

This work uses the LambdaMART implementation provided by LightGBM [16]. LightGBM is a gradient-boosted decision tree framework designed for efficient training on large datasets through histogram-based split finding, leaf-wise tree growth, and optimized memory usage. In addition to computational efficiency, it provides native support for learning-to-rank objectives, including LambdaMART with direct optimization of ranking metrics such as NDCG.

Tree-based models also naturally capture feature interactions, require minimal feature preprocessing, and handle heterogeneous numerical and categorical features effectively. As a result, LightGBM LambdaMART is used as the learning algorithm throughout this work.

3 RELATED WORK

We position our work against five fields of prior work: learning-to-rank algorithms, e-commerce ranking, position-bias correction, recommender-system personalization, and two-sided marketplace ranking.

Learning-to-rank and gradient-boosted rankers. Early learning-to-rank research established pointwise, pairwise, and listwise formulations for supervised ranking [18]. RankNet introduced a differentiable pairwise preference loss [3], while the progression from LambdaRank to LambdaMART showed that ranking metrics such as NDCG can be optimized indirectly by weighting pairwise gradients according to the metric change induced by swapping two documents [4]. LambdaMART combines this idea with gradient-boosted decision trees and has remained a strong baseline on standard LTR benchmarks such as LETOR [22] and the Yahoo! Learning to Rank Challenge [5]. Scalable gradient-boosting frameworks such as XGBoost [6] and LightGBM [16] made boosted-tree ranking practical for large tabular datasets, while neural ranking frameworks such as TF-Ranking [21] target similar objectives with differentiable architectures. In our work, we adopt a LightGBM LambdaMART ranker because the available signals are heterogeneous tabular features, the labels are sparse, and the resulting model remains relatively efficient and auditable compared with neural alternatives.

Learning-to-rank in e-commerce and marketplace search. LTR has been widely applied beyond web search to product search and recommendation, where relevance is usually inferred from implicit engagement such as clicks, add-to-carts, and purchases rather than editorial labels. Karmaker Santu et al. [15] show that e-commerce ranking differs from generic information retrieval because product ranking must account for purchase intent, popularity, price, availability, and catalog attributes. Industrial systems follow a similar pattern: Airbnb’s search ranking system uses large-scale behavioral logs to optimize marketplace search [9], while Prod2Vec demonstrates how product co-occurrence sequences can improve product representations for ranking and recommendation [8]. However, most of this work is validated in consumer-facing settings. In contrast, the present setting requires ranking around organizational procurement behavior rather than individual preference. This motivates an impression-level formulation tailored to wholesale interactions.

Position bias and unbiased learning-to-rank. Logged behavioral data is affected by position bias: items shown higher in a list receive more exposure and therefore more clicks, independently of their true relevance [13]. Counterfactual LTR methods address this by applying inverse-propensity weighting to clicks [14], estimating propensities from click models without explicit randomization [27], or jointly learning the ranker and propensity model [1]. These methods are most directly suited to click-dominated settings where position is the main confound. In our dataset the strongest labels are add-to-cart and purchase events, which are less frequent

but more commercially meaningful than clicks. We therefore use lighter-weight controls motivated by this literature: graded relevance labels that assign greater gain to stronger actions, and a position-weighted evaluation metric with propensities estimated only from the training split. This mitigates exposure bias in evaluation without claiming to solve unbiased LTR fully.

Personalization signals from recommender systems. Personalized ranking relies on estimating affinity between a buyer and an item. Recommender-system research provides several ways to construct such signals: matrix factorization models long-term user-item preference [17], Bayesian personalized ranking optimizes pairwise preference from implicit feedback [24], and session-based models capture short-term intent from recent browsing behavior [10, 26]. Embedding methods further compress behavioral and semantic similarity into dense representations. Item2Vec and Prod2Vec adapt word2vec-style co-occurrence learning to products [2, 8, 20], while sentence transformers encode product text into a semantic vector space [23]. We use these ideas as features rather than as a standalone recommender: buyer-product, buyer-category, session, behavioral-embedding, and semantic-embedding signals are fed into the LTR model so that ranking remains listwise and context-dependent.

Two-sided marketplace objectives. Marketplace ranking can also be viewed as a two-sided optimization problem, where the platform must balance buyer relevance with supplier exposure. Prior work formalizes such trade-offs through fairness of exposure [25] and marketplace ranking objectives that combine relevance, fairness, and stakeholder satisfaction [19]. These concerns are relevant in wholesale marketplaces because supplier visibility has direct commercial consequences. However, exposure balancing requires a reliable relevance model as a prerequisite. Therefore we focus on learning a stronger buyer-side relevance ranker and treat supplier-side fairness and exposure allocation as future work.

Taken together, prior work establishes the core components used in our work: LambdaMART ranking, boosted-tree implementations, graded implicit-feedback labels, position-bias controls, and personalization features. What remains less explored is their combination in the B2B wholesale setting defined in Section 2.1. This work addresses that gap by adapting established ranking methods to an impression-level formulation and evaluating them against the logged production order.

4 METHOD

This section describes the end-to-end methodology used to develop and evaluate the proposed learning-to-rank model. We first describe how the impression-level dataset is constructed from raw behavioral event logs and define the training, validation, and test splits. We then analyze key characteristics of the data through exploratory analysis, engineer point-in-time features, and define the graded relevance labels used for training. Next, we describe the selection of the label-gain mapping and the hyperparameter tuning procedure for the LightGBM LambdaMART model. Finally, we present the experimental protocol, including the baselines, evaluation metrics, and statistical comparison methodology used throughout the evaluation.

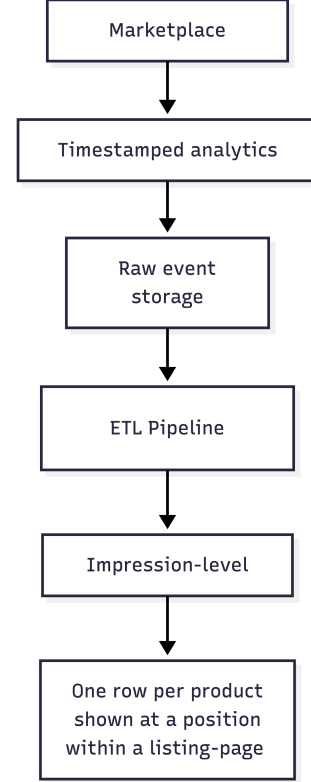


Figure 2: Construction of the impression-level dataset from raw behavioral event logs.

4.1 Dataset Construction

The dataset construction process is summarized in Figure 2. We construct the dataset from event-level behavioral tracking data collected on the marketplace. Interactions such as listing-page views, clicks, add-to-cart events, and purchases are logged as timestamped analytics events. These raw events are stored in the data warehouse and transformed through a scheduler-based data modeling pipeline into an impression-level dataset. Each row captures a single product shown at a specific position within a single listing-page view, restricted to logged-in buyers. Anonymous sessions are excluded because no stable buyer identity can be attached. Table 2 summarizes the scale of the assembled dataset.

Table 2: Scale of the impression-level dataset (full archive).

Metric	Value
Total impressions	~506M
Distinct list views	~24M
Unique logged-in buyers	~163K
Unique products (GTINs)	~450K
Unique sessions	~2.1M
Date range	Feb 2023 – present
Null session rate	~7%

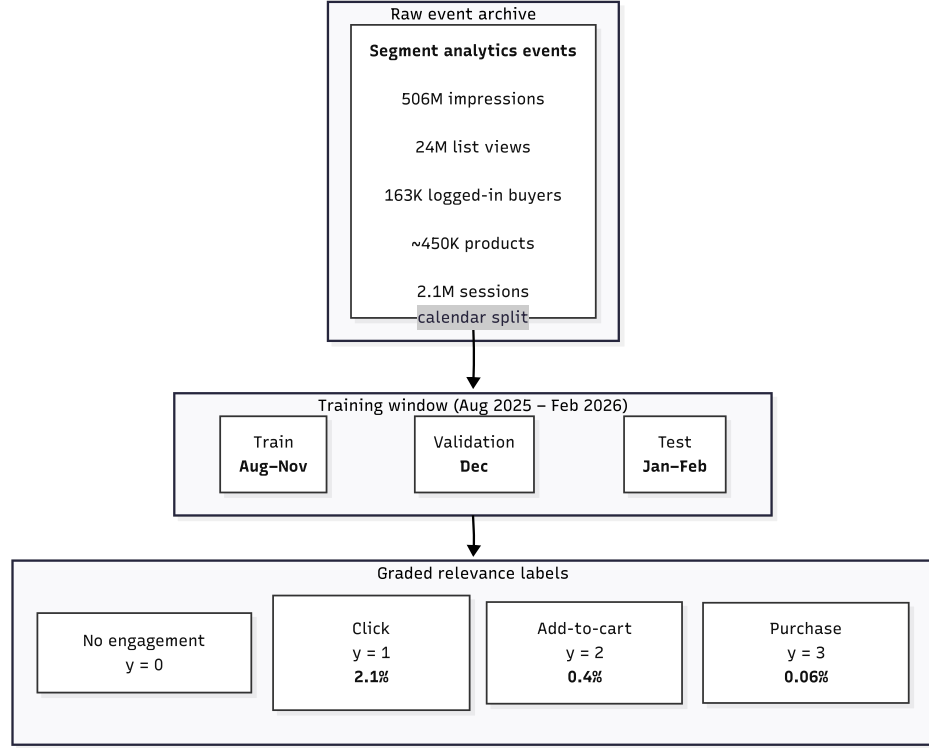


Figure 3: Overview of how the splits are defined and the sparsity of target labels.

As depicted in Figure 3 we consider a seven-month window (August 2025-February 2026) for training, over which the cart-traced purchase signal is complete, partitioned as train (August-November 2025), validation (December 2025), and test (January-February 2026). Table 3 gives the resulting split sizes and label sparsity.

Table 3: Split sizes and label sparsity. Positive LVs contain at least one observed engagement. Counts are rounded; M denotes millions and k denotes thousands.

Split	Rows M	LVs M	Pos. LVs	Clicks k	ATC k	Buy k
Train	130.0	6.64	23.0%	2,765	330	74
Val.	24.5	1.29	20.6%	482	58	14
Test	52.4	2.81	22.6%	1,101	139	33

Target Variables. We define three binary outcome signals, each utilizing a unique attribution logic that mirrors its underlying quality.

- **Click** (2.1% rate): This label represents the most immediate form of engagement. It is considered high quality because it requires a direct match between the user, the product, and the specific page environment where the impression occurred. This ensures the action is closely linked to the intent generated by the listing itself.
- **Add-to-cart** (0.4% rate): Identifies when a buyer expresses clear purchase intent by adding a product to their cart. It is

noted as medium quality because, while it confirms a strong interaction, the attribution is limited to the immediate visit to avoid including incidental re-encounters or discovery through other site features.

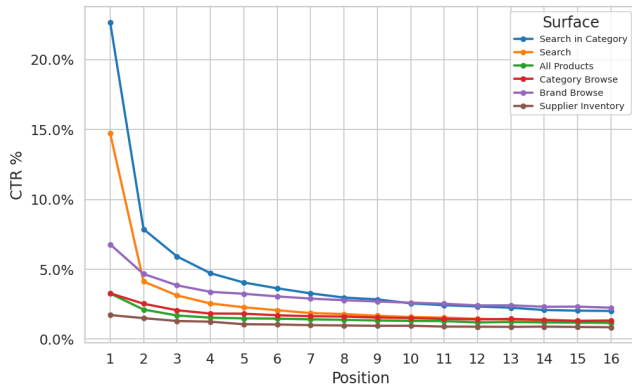
- **Purchase** (0.06% rate): Represents the ultimate conversion. It is considered high quality because it is verified through a direct sequence from the initial click to a finalized order. This deterministic approach ensures that the purchase label is only applied when the specific interaction leads directly to a completed transaction for that item.

4.2 Exploratory Data Analysis

Before specifying features, we inspect how buyers interact with the listing pages assembled above. The most consequential pattern is *position bias*: products shown near the top of a page receive far more clicks than equally relevant items lower down, regardless of surface. Figure 4 plots click-through rate by list position; CTR falls roughly sevenfold from position 1 to position 16, with the steepest decay on search surfaces. The decay shape is qualitatively similar across brand, category, and supplier-inventory browse, though absolute rates differ—search carries the highest engagement, supplier inventory the lowest. This confound between rank and relevance (Section 2.5) motivates graded labels and position-debiased W-NDCG@10 in evaluation.

Table 4: The twelve feature families that partition the 78-feature champion set, with one representative feature and the modeling hypothesis behind each family.

ID	Family	Hypothesis
L01	Product state & popularity	Popular and well-stocked products attract more clicks independently of who the buyer is. Popularity is defined by prior click-through and impression-level at the product level.
L02	Buyer $\times$ product affinity	A buyer re-orders products they have purchased before; recency and volume quantify the strength of that preference.
L03	Buyer $\times$ category affinity	Category-level history substitutes for product-level signal when a specific product is new to the buyer.
L04	Buyer order history	High-activity buyers engage more broadly; lifetime spend and recency index overall procurement intensity.
L05	Buyer profile & firmographics	Firm size, territory, and commercial profile correlate with category and price-point preferences.
L06	Session context	In-session engagement reveals intent for the current browsing episode and adjusts for fatigue or exploration.
L07	Cross-session user priors	Long-run per-user engagement rates distinguish habitual browsers from transactional buyers.
L08	Surface & time context	Relevance signals differ by entry point; search implies explicit intent whereas category browse implies exploration.
L09	In-list relative standing	A product's rank among co-retrieved competitors matters more than its absolute price or stock level.
L10	Target-encoded taxonomy	Smoothed brand- and category-level click and conversion rates provide a generalizing prior for thin-history products.
L11	Behavioral embedding sim.	Products that co-occur in past sessions are likely co-relevant; word2vec cosine similarity captures latent substitutability.
L12	Semantic embedding sim.	Products with similar titles are semantically interchangeable; sentence-transformer cosines complement behavioral co-occurrence.

**Figure 4: Click-through rate by list position and listing surface. CTR drops roughly 7 $\times$ from position 1 to position 16; decay is consistent across surfaces but varies in steepness.**

4.3 Feature Engineering

We describe each impression by 78 features, all computed *strictly point-in-time*, so every value is materialized as it would have been known at the moment the product was shown. This prevents the impression's own outcome, or any later event, from leaking into the training features. The modeling unit is a product impression within a list view; supplier offers are not ranked separately, but are

summarized by product-level offer aggregates such as active offer count, minimum price, price spread, and total inventory. We group the features into twelve broad families by the question they answer. Table 4 summarizes the families with one example and hypothesis each; the complete enumeration and exact point-in-time definitions are deferred to Appendix C.

Product state and popularity. Per-product point-in-time signals (historical click/add-to-cart rates, prior impressions, live offers, minimum price, inventory) plus smoothed brand and category level quality, capturing how desirable and available a product is, with a generalizing prior for sparse-history products.

Buyer affinity and identity. The buyer's prior interaction with the *same* product (orders, spend, recency), with its *category*, and overall (lifetime orders, spend, recency), plus stable firmographics (country, territory, firm size, persona).

Session and surface context. In-session behavior so far (list views, clicks, add-to-carts), longer-run per-user engagement intensity, and the view's own context (search / brand / category / supplier-inventory flags, hour, day of week), so the same product can rank differently by where, when, and to whom it is shown.

Listwise relative standing. Within each list view, the percentile rank of a product's price, offer count, inventory, and historical rates among its co-retrieved competitors, letting the listwise objective

judge a product against the items it actually competes with rather than by absolute magnitudes.

Semantic and behavioral similarity. Cosine similarities between the candidate and the buyer’s prior clicks, add-to-carts, purchases, and category history, from *word2vec* vectors of product co-occurrence (behavioral) and transformer-based vectors of titles (semantic), capturing which products are browsed together or are close in semantic space.

Representation choices. High-cardinality brand/category IDs are replaced by out-of-fold smoothed target encodings (avoiding rare-level overfit and leakage), bucketed buyer attributes are mapped to ordinal codes so trees split on their natural ordering, and the logged page number is dropped as a position proxy. The embedding features are enumerated in Appendix C.

4.4 Leakage prevention

The main leakage risk is not the tree model itself, but transformations that estimate statistics from outcomes. We therefore separate three cases. First, all warehouse aggregates are materialized with event-time predicates, so product, buyer, category, and session counts exclude the current impression outcome and any later behavior. Second, brand and category target encodings are fitted only on the training split: training rows receive five-fold out-of-fold encodings at the LIST_VIEW_ID level, while validation and test rows are mapped using encoders fitted on the full training split. Thus no validation or test labels enter either model selection or final evaluation features. Third, behavioral word2vec embeddings are trained from training-split list views only; downstream validation/test embedding-similarity features use the frozen vectors and each row’s prior buyer history. Title embeddings use a fixed text encoder and do not use engagement labels.

Table 5: Candidate training label-gain maps in the development-set sweep. Selection uses fixed validation NDCG@10 with evaluation gain $g_{\text{eval}}(y) = 2^y - 1 = [0, 1, 3, 7]$.

ID	$g_{\text{train}}(y)$ for $y = 0, 1, 2, 3$	Shape
B1	[0, 1, 3, 7]	purchase-steep (default)
B2	[0, 1, 2, 3]	linear (selected)
B3	[0, 1, 3, 15]	purchase-steep (steeper)
B4	[0, 0, 3, 7]	click-suppressing
B5	[0, 0, 0, 1]	purchase-only (binary)

4.5 Label-Gain Assignment

LambdaMART does not treat the labels $y \in \{0, 1, 2, 3\}$ as ordinary class numbers. It first maps them to gains $g(y)$, so a swap that moves a purchase-labeled item above a click-labeled item can be made much more valuable than a swap between a click and a non-click. The gain map therefore determines how strongly the training objective values clicks, add-to-carts, and purchases relative to one another.

LambdaMART optimizes NDCG under an explicit *training gain map* $g_{\text{train}}(y)$ that converts each graded relevance level $y \in \{0, 1, 2, 3\}$ (no engagement, click, add-to-cart, purchase) into a numeric gain.

Because this gain enters the objective used to construct pairwise lambdas, it is a modeling choice rather than a fixed constant, and we select it empirically. To keep the sweep comparable, only g_{train} changes across candidates; validation is always computed with the fixed evaluation gain

$$g_{\text{eval}}(y) = 2^y - 1, \quad y \in \{0, 1, 2, 3\},$$

so the same NDCG yardstick is used for every candidate. Starting from the library default [0, 1, 3, 7] (steeply purchase-weighted), we swept five candidate maps spanning three shapes — purchase-steep, linear, and click-suppressing, as listed in Table 5 — with all other settings held fixed. Each candidate was trained on the development train split and scored on validation; the map was chosen by the highest validation NDCG@10. The linear [0, 1, 2, 3] map was selected and is used by every subsequent model.

4.6 Hyperparameter Tuning

With the gain map fixed, we tuned the LightGBM ranker on a 10% subset of the training dataset maximizing validation NDCG@10 with per-trial pruning on the validation metric. Table 6 defines the search space, which covers the learning rate, tree capacity (leaves and minimum samples per leaf), the feature- and row-subsampling fractions and bagging frequency, and L1/L2 leaf regularization; the label gain, the number of boosting rounds (early-stopped on validation), and the seed are held fixed. We ran 30 trials and retrained the best configuration, with selection again restricted to the validation split. Tuning yielded a modest improvement over the hand-tuned defaults (reported in Section 5); the resulting configuration is the one carried into the full-scale runs.

Table 6: Optuna search space (TPE, validation NDCG@10) and the selected configuration.

Hyperparameter	Search space	Selected
learning_rate	[0.02, 0.10], log	0.033
num_leaves	{31, 63, 95, 127}	127
min_child_samples	[50, 300]	168
feature_fraction	[0.7, 1.0]	0.74
bagging_fraction	[0.7, 1.0]	0.91
bagging_freq	{1, 3, 5, 7}	5
lambda_l1	[0.0, 5.0]	2.61
lambda_l2	[0.0, 10.0]	4.28

4.7 Full Scale Experiment Setup

In this section we fix the metrics and the comparison protocol.

Baselines. We compare against the *logged production order* and *Random*. This baseline is exactly the order in which products were shown to buyers in production, i.e. sorting each list view by the logged POSITION. It is the output of Qogita’s production ranking stack during the test window: a TOPSIS-style product scorer combined with surface-specific retrieval, search relevance, availability, inventory, and tie-breaking rules. We report it as the logged production order rather than a clean reimplement of TOPSIS alone. Random is a uniform within-list shuffle indicating the no-information floor.

Metrics. Ranking quality is reported on the held-out test split as macro-averages: each list-view receives a per-group score and we report the arithmetic mean over groups. For NDCG-style metrics, list-views whose candidate set contains no positive label have IDCG@ $k = 0$ and are therefore undefined; we exclude these groups from the NDCG denominator rather than assigning an arbitrary zero. Hit@ k and MRR@10 use the full list-view denominator, assigning zero when no relevant item is retrieved in the evaluated prefix. This means NDCG values answer “how well are positives ordered when a positive exists”, whereas Hit@ k reports the unconditional probability of surfacing an observed positive in the top k .

- **NDCG@10** — primary metric, at the production list depth, with exponential gains $2^y - 1$.
- **W-NDCG@10** — position-debiased NDCG@10. On training impressions only we estimate an engagement propensity π_p per list position p (the smoothed fraction of rows with any engagement, normalized by the top-five plateau and clipped to $[\pi_{\min}, 1]$ with $\pi_{\min} = 0.05$); each test row inherits an inverse-propensity weight $w_i = \min(1/\pi_{p(i)}, 1/\pi_{\min})$. Weighting is row-level, and the reported metric is group-level.
- **MRR@10** — mean reciprocal rank of the first item with $y > 0$ in the top 10. Larger is better.
- **Hit@5 / Hit@5_ATC / Hit@5_BUY** — fraction of list-views with at least one click / add-to-cart / purchase in the top 5.

NDCG@5 and NDCG@24 are also tracked. Statistical comparison resamples list-views with replacement ($B=2000$ draws), recomputes the macro-mean of the per-group metric on each draw, and reports a 95% CI from the 2.5th and 97.5th percentiles. Paired comparisons subtract baseline per-group scores from candidate scores before bootstrapping. For full-scale training we use a machine with 30 vCPU and 230 GiB RAM running on CPU only.

5 EVALUATION

We evaluate the hyperparameter-tuned 78-feature LightGBM LambdaMART model with label gain $[0, 1, 2, 3]$ and the configuration of Section 4.6 on two test scopes. The *full test window* (the entire 52M-row Jan–Feb 2026 split, ~ 2.8 M list-views) carries the headline comparison against the production baselines with paired-bootstrap confidence intervals (Section 5.1.2) and the data-scaling study (Section 5.1.1). The *development subset* (the deterministic 10%/5%/5% sample) carries the per-segment breakdowns i.e. surface and cold-start (Section 5.1.3) where scoring baselines and bootstrap CIs for every slice are cheap to compute. Absolute NDCG is comparable *within* a scope but not across them, because the two use different test samples and embedding coverage; every comparison we draw stays inside one scope.

5.1 Results

Results are presented in three parts: the data-scaling study used to select the operating point, the full-scale comparison against the production baselines, and robustness analyses across listing surfaces and cold-start scenarios.

5.1.1 Data-scaling study. Because the dev-phase tuning used a 10% sample, we ask where the lift saturates. We trained the champion at

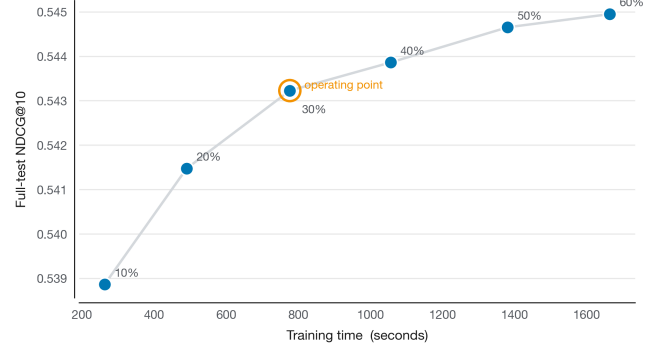


Figure 5: Cost–quality trade-off: full-test NDCG@10 vs. training time. Gain in quality provides diminishing returns while training cost keeps rising; the circled point is the 30% operating point.

10 to 60% of the 130M-row dataset, each scored on the identical full test window. As shown in Figure 5, the curve indicates the diminishing returns in the relative NDCG@10 gain per 10-point increment which collapses from $+0.48\%$ ($10 \rightarrow 20\%$) to $+0.05\%$ ($50 \rightarrow 60\%$), inside the 0.5% threshold the sweep was designed to detect. We take **30% as the operating point**: it captures essentially all of the achievable quality at a fraction of the training cost, which grows roughly linearly with rows (Figure 5).

Table 7: Full-scale comparison on the held-out test window. Champion = tuned 78-feature LightGBM LambdaMART at 30% train. Logged = logged production order. NDCG metrics exclude all-zero list-views with IDCG = 0; Hit@5 and MRR@10 use all list-views. Δ is champion–logged order in percentage points; CIs are paired-bootstrap 95% intervals.

Metric	Rnd.	Logged	Champ.	Δ pp	95% CI
NDCG@5	0.286	0.389	0.482	+9.30	[9.20, 9.41]
NDCG@10	0.351	0.448	0.543	+9.55	[9.45, 9.64]
NDCG@24	0.470	0.543	0.608	+6.51	[6.43, 6.58]
W-NDCG@10	0.348	0.429	0.537	+10.77	[10.67, 10.86]
MRR@10	0.071	0.090	0.113	—	—
Hit@5	0.102	0.124	0.156	+3.27	[3.24, 3.31]

5.1.2 Full-scale comparison against the baselines. Table 7 reports the operating-point champion (30% train; Section 5.1.1) against the logged production order and Random on the *entire* 52M-row test window, with paired-bootstrap 95% CIs. The champion improves NDCG@10 by **9.55 pp** over the logged production order (0.543 vs. 0.448) and W-NDCG@10 by **10.77 pp** (0.537 vs. 0.429); Figure 6 confirms that every interval clears zero by a wide margin. The W-NDCG lift exceeding the unweighted lift indicates the model is moving relevant items from deeper positions into the top 10, which the propensity-weighted metric credits more heavily. The lift is unambiguous over 2.8M list-views, which makes it credible.

Tuning carries to full scale. The label gain and hyperparameters were selected on the development split, so we confirm they still help at scale. As shown in Figure 7 the tuned configuration

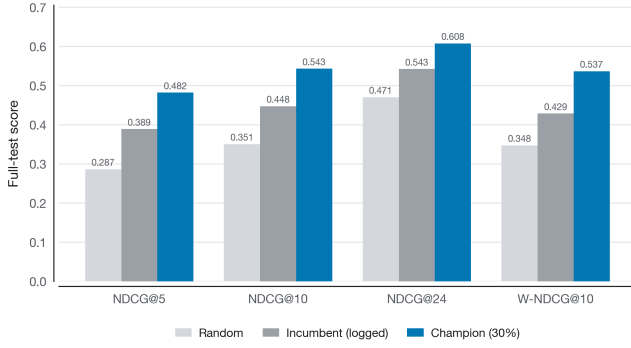


Figure 6: Full-scale baseline comparison on the entire test window. The champion exceeds the logged production order and Random on every metric.

beats the fixed default on full-test NDCG@10 by +0.13 to +0.20 pp at every matching fraction, showing that the dev-slice search was not an artifact of the 10% sample.

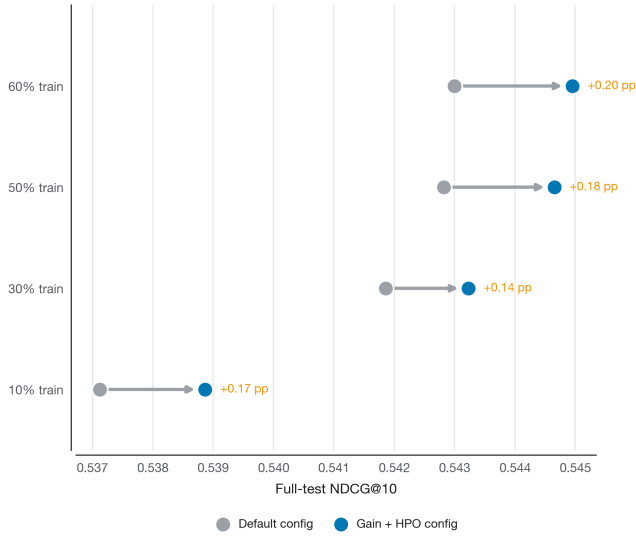


Figure 7: Default vs. gain + HPO configuration, scored on the identical full test window at each shared training fraction. The tuned config wins throughout; labels give the paired lift.

5.1.3 Per-surface and cold-start robustness. To see *where* the lift comes from — and whether it is carried by one easy surface — we break the comparison down on the development subset, where baselines and bootstrap CIs per slice are cheap. This is development scale (the full-window run is overall-only), but the two scopes are consistent: the logged production order’s per-surface NDCG@10 matches across them.

As seen in Figure 8, the lift holds on all four catalog surfaces, and its size tracks how much structure the logged production order already imposes. On SEARCH, where query matching gives the logged production order a strong order, the champion adds 5.0 pp;

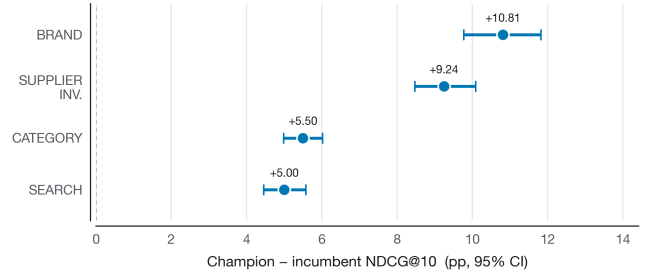


Figure 8: Per-surface champion–logged order NDCG@10 lift with paired-bootstrap 95% CIs. Largest where the logged production order is weakest (BRAND, SUPPLIER_INVENTORY), smallest on SEARCH.

on BRAND and SUPPLIER_INVENTORY, where the logged production order barely separates products, it adds 9.2–10.8 pp. The model gains most precisely where the logged production order is least informative, and every per-surface interval excludes zero.

Figure 9 shows each cold-start definition as a pair of segments: a colder segment with little or no history and a warmer segment where the model has more prior signal. The plotted value is the champion-minus-logged-order NDCG@10 lift, so bars above zero mean that the learned ranker improves over the logged production order for that segment. Across repeat purchase of the same GTIN, prior category history, product exposure, embedding-vocabulary coverage, and buyer lifetime orders, every confidence interval clears zero. The lift is larger where history is available (e.g. +10.8 pp for repeat purchasers vs. +5.5 pp cold), but cold buyers and out-of-vocabulary items still gain 5–8 pp: the ranker degrades gracefully where personalization signal is sparse rather than collapsing to the logged production order.

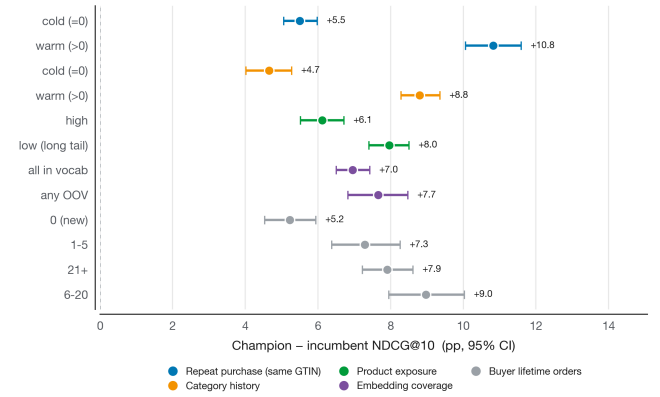


Figure 9: Champion–logged order NDCG@10 lift with paired-bootstrap 95% CIs across five cold-start segments. Every segment has positive gain. Warm segments gain more, but cold ones still gain 5–8 pp.

6 DISCUSSION

This thesis studies learning-to-rank in a setting that differs from the consumer benchmarks on which most ranking systems are

evaluated. In Qogita’s B2B marketplace, buyers are organizations, conversions are rare and delayed, and the same product may be listed by competing suppliers. The three research questions ask whether a personalized ranker can still be trained and evaluated under these conditions. The results suggest that it can.

6.1 RQ1: Does a LambdaMART ranker improve over the logged production order?

The full-scale comparison in Table 7 and Figure 6 answers RQ1 directly. On the entire 52-million-row test window, covering roughly 2.8 million list views, the champion improves NDCG@10 by 9.55 pp over the logged production order (0.543 against 0.448), with a paired-bootstrap 95% confidence interval of [9.45, 9.64]. The W-NDCG@10 lift is even larger at 10.77 points, which suggests that the model is not only improving already-visible top results but also moving relevant products up from deeper positions that receive less exposure under the logged production order.

The surface breakdown in Figure 8 shows that the lift is not carried by one easy surface. It is smallest on SEARCH, where keyword matching already gives the logged production order a reasonable starting order, and larger on BRAND and SUPPLIER_INVENTORY, where the logged production order has less information for separating products and often falls back on weak tie-breakers. This pattern supports the interpretation that the learned ranker adds buyer and product signal where the production scorer is least expressive, rather than simply reproducing the logged production order.

Figure 9 adds the cold-start view. Across repeat-purchase history, category history, prior product exposure, embedding-vocabulary coverage, and buyer lifetime orders, every segment remains positive and every confidence interval excludes zero. Warm segments gain more—around 10.8 points for repeat purchasers—but cold buyers and out-of-vocabulary products still improve by roughly five to eight points. The model therefore degrades gracefully toward non-personalized popularity and product-quality signals when personalization evidence is sparse, rather than collapsing back to the logged production order. This property matters for deployment: a buyer the model has never seen is still better served by the ranker than by the logged production order.

6.2 RQ2: Which label-gain map produces the best ranking, and does the selection transfer to full scale?

The gain sweep in Table 5 answers the first part of RQ2. Starting from the LightGBM default purchase-steep map [0, 1, 3, 7], we tested five candidate mappings and selected the linear map [0, 1, 2, 3] by validation NDCG@10. This result is consistent with the label distribution: purchases are the most valuable action, but they are too rare to dominate the training signal. The linear map gives clicks, add-to-carts, and purchases usable gradient signal instead of concentrating most weight on the small minority of query groups containing purchases.

The transfer part of RQ2 is supported by Figure 7. The selected gain map is carried into training fractions from 10% to 60% of the 130-million-row training corpus and scored on the same full test window. The tuned configuration remains ahead of the default at

every fraction, which supports using the selected gain map at full scale rather than re-selecting the training objective separately for each training size.

6.3 RQ3: Does a hyperparameter configuration tuned at development scale transfer to full-scale training?

RQ3 asks whether the 30-trial configuration selected on the 10% development sample still helps when the model is trained on larger fractions of the corpus.

Figure 7 shows that it does. At every matching training fraction from 10% to 60%, the gain + HPO configuration beats the fixed default on the identical full-test window by 0.14 to 0.20 pp NDCG@10. The gain is modest, but its consistency across scale suggests that the development-phase search did not merely overfit the 10% sample. Repeating the same hyperparameter search separately at each training fraction would therefore add cost without clear quality benefit.

6.4 Limitations

Several limitations bound the claims made in this work and should be noted alongside the positive results.

Offline evaluation only. Every metric reported here is computed against logged behavior under the logged production order’s own exposure policy. The model is evaluated on how well it reorders what buyers were historically shown, not on how buyers respond to orderings they have never seen. The position-debiasing in W-NDCG reduces this gap but does not close it. The lift of 9.55 percentage points on NDCG@10 is a strong offline signal, but whether it translates to an equivalent gain in live click-through rate or order volume can only be determined by a controlled online experiment.

Per-segment results are at development scale. The per-surface and cold-start breakdowns reported in Section 5.1.3 are computed on the development subset rather than the full test window. The headline comparison is at full scale, and the two scopes are consistent where they overlap, but the segment-level confidence intervals should be treated as indicative rather than definitive until the full-window breakdown is run.

Single marketplace and training window. The dataset covers one B2B wholesale marketplace over seven months. Seasonal patterns outside that window, catalog categories not well represented in the training period, and buyer behaviors specific to Qogita’s market may not generalize to other B2B platforms. The training window is long enough to capture multiple reorder cycles for most buyers, but a longer history would strengthen the affinity and session-prior features for low-frequency buyers.

Cold-start for new products. The feature pipeline relies on historical impression and purchase data to compute product popularity, buyer affinity, and embedding similarity scores. A product that has never appeared in a listing page has no history for any of these features and falls back entirely on the target-encoded brand and category priors. The cold-start segmentation in Section 5.1.3 addresses buyer cold-start but does not separately evaluate product cold-start, which is a distinct failure mode for a catalog of 450K total products that turns over continuously.

The results support a practical framework for B2B ranking: build a point-in-time impression dataset, select graded gains empirically, and train a listwise LambdaMART model. The lift holds at full test scale, across catalog surfaces, and in cold-start slices, which suggests that the model is using buyer-affinity, session, and product-quality signals that the logged production order does not represent.

The remaining uncertainty is online behavior. Offline evaluation shows that the ranker better orders historically shown products, but only a live deployment can measure whether buyers respond differently to the reranked lists.

7 CONCLUSION

We asked whether personalized ranking can be made to work in a B2B wholesale marketplace, where buyers are organizations, conversions are rare and delayed, and products are listed through competing supplier offers rather than a single catalog entry.

The first contribution is the dataset and pipeline needed to study that question. From production real-time events, we assembled a seven-month impression-level dataset covering 163,000 logged-in buyers and 450K products. Each feature is materialized point-in-time through a scheduled ETL pipeline, with a cart-trace attribution chain that links listing impressions to downstream purchases. This gives the ranker graded relevance labels at click, add-to-cart, and purchase level without allowing future behavior to leak into the training rows.

The second contribution is the ranking model and tuning protocol. We trained a LightGBM LambdaMART ranker over 78 point-in-time features covering product popularity, buyer-product and buyer-category affinity, session context, in-list relative standing, target-encoded taxonomy, and behavioral and semantic embedding similarities. The label-gain map and hyperparameters were selected on a development subset, then carried to larger training fractions to test whether those choices transfer beyond the small tuning sample.

The offline evidence is strong. On the full 52-million-row test window, the champion improves NDCG@10 by 9.55 percentage points over the logged production order. The lift appears across all four catalog surfaces, is largest where the logged production order has the weakest ranking signal, and remains positive in cold-start segments with sparse buyer or product history. The development-scale tuning configuration also improves full-test NDCG@10 by 0.14 to 0.20 percentage points over the default at every training fraction from 10% to 60%, showing that a single dev-phase search is sufficient for this corpus and configuration space.

The limitations point to the next research questions. This thesis evaluates logged behavior from one marketplace, so the result should be tested with online experiments and replicated on other B2B catalogs before broader claims are made. Future work should also study stronger counterfactual learning methods for position bias, dedicated product cold-start evaluation, and two-sided ranking objectives that account for supplier exposure as well as buyer relevance. The key takeaway is that under offline evaluation, a point-in-time B2B impression dataset and a listwise personalized ranker can substantially improve over the logged production order.

REFERENCES

- [1] Qingyao Ai, Keping Bi, Cheng Luo, Jiafeng Guo, and W. Bruce Croft. 2018. Unbiased Learning to Rank with Unbiased Propensity Estimation. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*. 385–394.

- [2] Oren Barkan and Noam Koenigstein. 2016. Item2Vec: Neural Item Embedding for Collaborative Filtering. In *IEEE 26th International Workshop on Machine Learning for Signal Processing (MLSP)*. 1–6.
- [3] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. 2005. Learning to Rank Using Gradient Descent. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*. 89–96.
- [4] Christopher J. C. Burges. 2010. *From RankNet to LambdaRank to LambdaMART: An Overview*. Technical Report MSR-TR-2010-82. Microsoft Research.
- [5] Olivier Chapelle and Yi Chang. 2011. Yahoo! Learning to Rank Challenge Overview. In *Proceedings of the Learning to Rank Challenge, JMLR Workshop and Conference Proceedings*, Vol. 14. 1–24.
- [6] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 785–794.
- [7] Expedia Group. 2013. Personalize Expedia Hotel Searches – ICDM 2013. Kaggle. <https://www.kaggle.com/c/expedia-personalized-sort>. Accessed 2026-07-04.
- [8] Mihajlo Grbovic, Vladan Radosavljevic, Nemanja Djuric, Narayan Bhamidipati, Jaikit Savla, Varun Bhagwan, and Doug Sharp. 2015. E-commerce in Your Inbox: Product Recommendations at Scale. In *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1809–1818.
- [9] Malay Haldar, Mustafa Abdool, Prashant Ramanathan, Tao Xu, Shulin Yang, Huizhong Duan, Qing Zhang, Nick Barrow-Williams, Bradley C. Turnbull, Brendan M. Collins, and Thomas Legrand. 2019. Applying Deep Learning to Airbnb Search. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1927–1935.
- [10] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-Based Recommendations with Recurrent Neural Networks. In *International Conference on Learning Representations (ICLR)*.
- [11] Ching-Lai Hwang and Kwangsun Yoon. 1981. *Multiple Attribute Decision Making: Methods and Applications: A State-of-the-Art Survey*. Lecture Notes in Economics and Mathematical Systems, Vol. 186. Springer Berlin Heidelberg. <https://doi.org/10.1007/978-3-642-48318-9>
- [12] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems (TOIS)* 20, 4 (2002), 422–446.
- [13] Thorsten Joachims. 2002. Optimizing Search Engines Using Clickthrough Data. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 133–142.
- [14] Thorsten Joachims, Adith Swaminathan, and Tobias Schnabel. 2017. Unbiased Learning-to-Rank with Biased Feedback. In *Proceedings of the 10th ACM International Conference on Web Search and Data Mining (WSDM)*. 781–789.
- [15] Shubhra Kanti Karmaker Santu, Parikshit Sondhi, and ChengXiang Zhai. 2017. On Application of Learning to Rank for E-Commerce Search. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 475–484.
- [16] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [17] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37.
- [18] Tie-Yan Liu. 2009. Learning to Rank for Information Retrieval. *Foundations and Trends in Information Retrieval* 3, 3 (2009), 225–331.
- [19] Rishabh Mehrotra, James McInerney, Hugues Bouchard, Mounia Lalmas, and Fernando Diaz. 2018. Towards a Fair Marketplace: Counterfactual Evaluation of the Trade-off Between Relevance, Fairness and Satisfaction in Recommendation Systems. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*. 2243–2251.
- [20] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. Distributed Representations of Words and Phrases and Their Compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 26.
- [21] Rama Kumar Pasumarthi, Sebastian Bruch, Xuanhui Wang, Cheng Li, Michael Bendersky, Marc Najork, Jan Pfeifer, Nadav Golbandi, Rohan Anil, and Stephan Wolf. 2019. TF-Ranking: Scalable TensorFlow Library for Learning-to-Rank. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 2970–2978.
- [22] Tao Qin, Tie-Yan Liu, Jun Xu, and Hang Li. 2010. LETOR: A Benchmark Collection for Research on Learning to Rank for Information Retrieval. *Information Retrieval* 13, 4 (2010), 346–374.
- [23] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. 3982–3992.
- [24] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback.

- In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*. 452–461.
- [25] Ashudeep Singh and Thorsten Joachims. 2018. Fairness of Exposure in Rankings. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 2219–2228.
- [26] S. Wang, L. Hu, Y. Wang, L. Cao, Q. Z. Sheng, and M. Orgun. 2019. Sequential Recommender Systems: Challenges, Progress and Prospects. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19)*. 6332–6338.
- [27] Xuanhui Wang, Nadav Golbandi, Michael Bendersky, Donald Metzler, and Marc Najork. 2018. Position Bias Estimation for Unbiased Learning to Rank in Personal Search. In *Proceedings of the 11th ACM International Conference on Web Search and Data Mining (WSDM)*. 610–618.

A ETHICAL CONSIDERATIONS

This work analyzes production behavioral data from logged-in business buyers. Buyer identity is represented by opaque account identifiers, no personally identifiable information is stored in the pipeline, and no data is shared outside Qogita.

The main residual risk is of over-interpreting offline evaluation. The dataset is proprietary and cannot be publicly released, but the quantitative claims are reproducible within the organization from the documented pipeline and versioned model artifacts.

B AI USAGE

Large language models were used for reframing the content during the preparation of this thesis manuscript. Specifically, Claude Sonnet 4.6 was used to assist in refining prose.

C FEATURE CATALOG

The champion model uses 78 features grouped into the twelve families summarized in Table 4. Every feature is computed strictly point-in-time: no value observes information that post-dates its impression timestamp.

PRODUCT_OFFER_COUNT_PIT Number of active offers for the product at impression time.

PRODUCT_INVENTORY_PIT Product-level inventory available at impression time.

PIT_NUM_OFFERS Point-in-time count of supplier offers for the product.

PIT_MIN_PRICE_EUR Lowest available offer price for the product in euros.

PIT_PRICE_SPREAD Difference between the highest and lowest active offer price.

PIT_TOTAL_INVENTORY Total active inventory across offers for the product.

PRIOR_IMPRESSIONS Number of earlier listing impressions for the product.

PRODUCT_HISTORICAL_CTR Historical product click-through rate before the impression.

PRODUCT_HISTORICAL_ATC_RATE Historical product add-to-cart rate before the impression.

LOG_PIT_MIN_PRICE_EUR Log-transformed point-in-time minimum offer price.

LOG_PIT_TOTAL_INVENTORY Log-transformed total point-in-time product inventory.

LOG_PRODUCT_INVENTORY_PIT Log-transformed point-in-time product inventory.

LOG_PRIOR_IMPRESSIONS Log-transformed count of prior product impressions.

PIT_REL_SPREAD Relative spread between active offer prices.

BP_PRIOR_ORDERS Buyer’s prior order count for the same product.

BP_PRIOR_UNITS Buyer’s prior units ordered for the same product.

BP_PRIOR_SPEND_EUR Buyer’s prior spend on the same product in euros.

BP_DAYS_SINCE_LAST_ORDER Days since the buyer last ordered the same product.

BP_AVG_UNIT_PRICE_EUR Buyer’s historical average unit price for the same product.

BC_PRIOR_ORDERS Buyer’s prior order count in the product category.

BC_PRIOR_UNITS Buyer’s prior units ordered in the product category.

BC_PRIOR_SPEND_EUR Buyer’s prior spend in the product category in euros.

BC_DAYS_SINCE_LAST_ORDER Days since the buyer last ordered from the product category.

BUYER_PRIOR_ORDER_COUNT Buyer’s total historical order count before the impression.

BUYER_PRIOR_ORDERLINE_COUNT Buyer’s total historical order-line count before the impression.

BUYER_PRIOR_ORDER_UNITS Buyer’s total historical units ordered before the impression.

BUYER_PRIOR_ORDER_SPEND_EUR Buyer’s total historical order spend before the impression.

BUYER_DAYS_SINCE_LAST_ORDER Days since the buyer’s most recent order.

BUYER_IS_STAFF Indicator that the buyer account is internal staff.

BUYER_COUNTRY Country associated with the buyer account.

BUYER_TERRITORY Commercial territory assigned to the buyer.

HUBSPOT_PERSONA_GROUP HubSpot persona segment for the buyer.

BUYER_SIZE_ORD Ordinal encoding of the buyer size bucket.

BUYER_NUMBER_OF_EMPLOYEES_ORD Ordinal encoding of the buyer employee-count bucket.

BUYER_ANNUAL_TURNOVER_ORD Ordinal encoding of the buyer annual-turnover bucket.

SESSION_LIST_VIEWS_SO_FAR Number of listing pages viewed earlier in the session.

SESSION_CLICKS_SO_FAR Number of product clicks earlier in the session.

SESSION_ATCS_SO_FAR Number of add-to-cart events earlier in the session.

SESSION_DISTINCT_PRODUCTS_SEEN Number of distinct products seen earlier in the session.

SESSION_DISTINCT_SURFACES Number of distinct listing surfaces used in the session.

SESSION_HAS_SEARCHED Indicator that the session has included a search action.

SESSION_DURATION_MINUTES Elapsed session duration in minutes at impression time.

SESSION_CLICK_RATE_SOFAR Session click rate before the current impression.

SESSION_ATC_RATE_SOFAR Session add-to-cart rate before the current impression.

USER_PRIOR_SESSIONS Number of earlier sessions for the buyer.

USER_PRIOR_CLICKS Number of earlier product clicks by the buyer.

USER_PRIOR_ATCS Number of earlier add-to-cart events by the buyer.

USER_CLICKS_PER_SESSION Buyer’s historical clicks per session.

USER_ATCS_PER_SESSION Buyer’s historical add-to-cart events per session.

FROM_SEARCH Indicator that the listing came from search.

FROM_CATEGORY Indicator that the listing came from category browse.

FROM_BRAND Indicator that the listing came from a brand page.

FROM_SUPPLIER_INVENTORY Indicator that the listing came from supplier inventory.

HOURL_OF_DAY Hour of day when the impression occurred.

DAY_OF_WEEK Day of week when the impression occurred.

IS_WEEKEND Indicator that the impression occurred on a weekend.

PRICE_RANK_IN_LIST Candidate’s within-list percentile rank by price.

OFFER_COUNT_RANK_IN_LIST Candidate’s within-list percentile rank by offer count.

INVENTORY_RANK_IN_LIST Candidate’s within-list percentile rank by inventory.

HIST_CTR_RANK_IN_LIST Candidate’s within-list percentile rank by historical CTR.

HIST_ATC_RANK_IN_LIST Candidate’s within-list percentile rank by historical add-to-cart rate.

BRAND_ID_TE_CTR Out-of-fold target encoding of brand click-through rate.

BRAND_ID_TE_ATC Out-of-fold target encoding of brand add-to-cart rate.

BRAND_ID_TE_BUY Out-of-fold target encoding of brand purchase rate.

BRAND_ID_TE_REL Out-of-fold target encoding of brand graded relevance.

CATEGORY_ID_TE_CTR Out-of-fold target encoding of category click-through rate.

CATEGORY_ID_TE_ATC Out-of-fold target encoding of category add-to-cart rate.

CATEGORY_ID_TE_BUY Out-of-fold target encoding of category purchase rate.

CATEGORY_ID_TE_REL Out-of-fold target encoding of category graded relevance.

EMB_CLICK_SIM Behavioral embedding similarity to products the buyer clicked.

EMB_ATC_SIM Behavioral embedding similarity to products the buyer added to cart.

EMB_BUY_SIM Behavioral embedding similarity to products the buyer purchased.

EMB_CAT_SIM Behavioral embedding similarity to the buyer’s category history.

EMB_IN_VOCAB Indicator that the product has a behavioral embedding vector.

TITLE_CLICK_SIM Title-embedding similarity to products the buyer clicked.

TITLE_ATC_SIM Title-embedding similarity to products the buyer added to cart.

TITLE_BUY_SIM Title-embedding similarity to products the buyer purchased.

TITLE_CAT_SIM Title-embedding similarity to the buyer’s category history.

D ARTIFACT APPENDIX

D.1 Abstract

The artifact is a Python codebase implementing the end-to-end personalized product ranking pipeline described in this thesis. It covers feature construction, data preparation, embedding generation, LambdaMART training, and offline evaluation. The unit test suite can be run without access to production data. The full feature pipeline and large-scale training require internal Qogita credentials and infrastructure and cannot be run externally.

D.2 Description and Requirements

D.2.1 Security, privacy, and ethical concerns. The training data is proprietary and contains behavioral records of Qogita buyers. It is not distributed with this artifact.

D.2.2 Hardware dependencies. Full-scale training (30% of the corpus, the operating point) requires a machine with at least 128 GiB RAM and 16 CPU cores. The Airflow DAG used for production runs provisions a 230 GiB, 30-vCPU Kubernetes pod. Unit tests and the OpenSearch proof-of-concept run on any modern laptop.

D.2.3 Software dependencies. Python 3.11, managed by Poetry. Key dependencies: lightgbm, optuna, gensim, sentence-transformers, snowflake-connector-python, duckdb, pyarrow. We additionally require Docker and Docker Compose to run a local OpenSearch 2.15 instance for testing the model in a simulated production setting.

D.3 Evaluation workflow

D.3.1 Major claims.

- (C1) The LambdaMART champion improves NDCG@10 by 9.55 pp over the logged production order on the full test window, with a paired-bootstrap 95% CI of [9.45, 9.64] that excludes zero. Reported in Table 7 and Figure 6.
- (C2) Quality saturates by 30% of the training corpus; the relative NDCG@10 gain per ten-point increment falls below 0.5% by the 50–60% step. Reported in Figure 5.
- (C3) The Gain and HPO configuration selected at development scale beats the default configuration on the full test window at every training fraction from 10 to 60%, by 0.14 to 0.20 pp. Reported in Figure 7.

D.3.2 Experiments.

- (E1) **Feature pipeline** [2 human-hours + up to 18 compute-hours + 200 GB disk]. Builds the Snowflake staging tables and streams monthly Parquet files to data/raw/.
Preparation: Snowflake credentials configured; warehouse with at least X-Large compute available.
Execution: Python script.
Results: Seven monthly parquet files.
- (E2) **Full-scale training and evaluation** [1 human-hour + up to 24 compute-hours + 50 GB disk]. Trains the champion at the 30% operating point and evaluates against baselines (C1).
Preparation: Split files from E1 are present; and embeddings are built.
Execution: python experiment script.
Results: JSON containing NDCG@5/10/24, W-NDCG@10, MRR@10, Hit@5, and paired-bootstrap CIs. NDCG@10 should be 0.543 ± 0.01 for the champion and 0.448 ± 0.01 for the logged production order.
- (E3) **Data-scaling study** [1 human-hour + 48 compute-hours]. Trains at six fractions and produces the cost-quality curve (C2) and tuning-transfer validation (C3).
Preparation: As for E2.
Execution: Fault-tolerant, resumable job run on the cloud.
Results: One metrics.json per fraction.